Building brainplots.py: a complete API reference for cortical visualization in Python

Every major Python neuroimaging visualization library now converges on a common workflow: load a surface mesh, attach vertex-wise data as overlays, render multi-view panels, and export at publication DPI. This reference provides exact, current API signatures and working code for surfplot (v0.2.0), nilearn (v0.13.1), PyVista (v0.47.1), netgraph (v4.13.2), mne-connectivity (v0.7.0), and networkx (v3.6.1) — everything needed to build a comprehensive `brainplots.py` module targeting Nature Neuroscience-grade figures.

---

1. surfplot delivers the fastest path to 4-view brain panels

surfplot (v0.2.0 stable, v0.3.0rc0 pre-release Dec 2024) PyPI wraps brainspace's VTK rendering into a matplotlib-friendly API. The library accepts file paths or BSPolyData objects Readthedocs and produces standard matplotlib Figures directly. Readthedocs

Plot() constructor — exact signature

```python
from surfplot import Plot

p = Plot(
    surf_lh=None,          # str | os.PathLike | BSPolyData — left hemisphere
    surf_rh=None,          # str | os.PathLike | BSPolyData — right hemispher
    layout='grid',         # 'grid' | 'column' | 'row'
    views=None,            # str | list — 'lateral','medial','dorsal','ventra
    mirror_views=False,    # bool — flip RH view order (row/column only)
    flip=False,            # bool — swap L/R display order
    size=(500, 400),       # tuple — VTK render resolution (not matplotlib fi
    zoom=1.5,              # float
    background=(1, 1, 1), # tuple — RGB background
    label_text=None,       # dict — keys: 'left','right','top','bottom'
    brightness=0.5         # float — gray surface brightness 0-1
)
```

The default `layout='grid'` with `views=None` (→ `['lateral', 'medial']`) produces exactly the standard **4-view panel**: lateral-LH, lateral-RH, medial-LH, medial-RH. Readthedocs

add_layer() — exact signature

```python
p.add_layer(
    data,                    # np.ndarray | dict | str | GiftiImage | Cifti2
    cmap='viridis',          # str | matplotlib colormap
    alpha=1,                 # float 0-1 (added v0.2.0)
    color_range=None,        # tuple (min, max) or None for auto
    as_outline=False,        # bool — draw region borders only
    zero_transparent=True,   # bool — 0-valued vertices → transparent
    cbar=True,               # bool
    cbar_label=None          # str — colorbar label
)
```

The `data` parameter accepts a **numpy array** of length equal to total vertices (e.g., **64,984** for fsLR 32k bilateral), Readthedocs a **dict** with `'left'`/`'right'` keys, or file paths to GIFTI/CIFTI. Readthedocs Readthedocs Setting `as_outline=True` renders only parcellation borders — essential for overlaying Schaefer or Yeo boundaries on statistical maps. Readthedocs GitHub

build() and colorbar customization

```python
fig = p.build(
    figsize=None,        # tuple — matplotlib figure size (inches)
    colorbar=True,       # bool
    cbar_kws=None,       # dict — see below
    scale=(2, 2)         # tuple — upscale factor for resolution
)
```

The `cbar_kws` dict controls all colorbar properties: `location` ('bottom'|'top'|'left'|'right'), `label_direction` (0=horizontal, 90=vertical), `n_ticks` (int), `decimals` (int), `fontsize` (int), `draw_border` (bool), `shrink` (float), `aspect` (int), `pad` (float), `fraction` (float), and `outer_labels_only` (bool). Readthedocs Readthedocs The returned object is a standard `matplotlib.figure.Figure`. Readthedocs

Loading surfaces — the three approaches

```python
# RECOMMENDED: neuromaps returns file paths (.gii)
from neuromaps.datasets import fetch_fslr
surfaces = fetch_fslr()
lh, rh = surfaces['inflated']       # file paths
sulc_lh, sulc_rh = surfaces['sulc'] # sulcal depth for shading

# ALT: brainspace returns BSPolyData objects directly
from brainspace.datasets import load_conte69
lh_mesh, rh_mesh = load_conte69()   # midthickness surface, 32492 verts/hemi

# ALT: neuromaps fsaverage
from neuromaps.datasets import fetch_fsaverage
surfaces = fetch_fsaverage(density='164k')
lh, rh = surfaces['inflated']
```

Critical vertex counts: fsLR 32k = **32,492 per hemisphere** (64,984 total). Without medial wall = **29,696 per hemisphere** (59,412 total) — use `surfplot.utils.add_fslr_medial_wall(data)` to pad back. Readthedocs fsaverage5 = **10,242 per hemisphere.**

Complete publication-quality example

```python
import numpy as np
from neuromaps.datasets import fetch_fslr
from surfplot import Plot
from surfplot.datasets import load_example_data
from brainspace.datasets import load_parcellation

surfaces = fetch_fslr()
lh, rh = surfaces['inflated']
sulc_lh, sulc_rh = surfaces['sulc']

p = Plot(lh, rh, layout='grid', views=['lateral', 'medial'],
         size=(500, 400), zoom=1.5)

# Layer 1: sulcal shading
p.add_layer({'left': sulc_lh, 'right': sulc_rh}, cmap='binary_r', cbar=False

# Layer 2: statistical overlay
lh_data, rh_data = load_example_data()  # Neurosynth z-scores
p.add_layer({'left': lh_data, 'right': rh_data}, cmap='YlOrRd_r',
            color_range=(2, 12), zero_transparent=True, cbar_label='Associat

# Layer 3: parcellation outlines
lh_parc, rh_parc = load_parcellation('schaefer')
p.add_layer({'left': lh_parc, 'right': rh_parc}, cmap='gray',
            as_outline=True, cbar=False)

fig = p.build(figsize=(8, 6), cbar_kws={
    'location': 'bottom', 'label_direction': 0, 'decimals': 1,
    'fontsize': 8, 'n_ticks': 3, 'shrink': 0.25, 'draw_border': False
})
fig.savefig('surface_4view.png', dpi=400, bbox_inches='tight',
            facecolor='white', edgecolor='none')
```

---

2. nilearn 0.13.1 overhauled its surface API around SurfaceImage

nilearn's PyPI surface plotting underwent a **major restructuring in v0.11.0** (Dec 2024): `SurfaceImage`, `PolyMesh`, and `PolyData` classes moved from `nilearn.experimental.surface` to `nilearn.surface` and `nilearn.datasets`. Neurostars The old `fetch_surf_fsaverage()` still works but returns file-path strings; the new `load_fsaverage()` returns `PolyMesh` objects. Nilearn

plot_surf_stat_map() — exact current signature

```python
nilearn.plotting.plot_surf_stat_map(
    surf_mesh=None,          # str | list[2 ndarray] | PolyMesh | None
    stat_map=None,           # str | ndarray | SurfaceImage | None
    bg_map=None,             # str | ndarray | SurfaceImage | None
    hemi='left',             # 'left' | 'right' | 'both'
    view=None,               # 'lateral','medial','dorsal','ventral','anteri
    engine='matplotlib',     # 'matplotlib' | 'plotly'
    cmap='RdBu_r',           # colormap
    colorbar=True,           # bool
    avg_method=None,         # 'mean','median','min','max' (default: 'mean'
    threshold=None,          # float | 'auto' | None
    alpha=None,              # float | 'auto' | None
    bg_on_data=False,        # bool — multiply stat by bg for joint display
    vmin=None, vmax=None,    # float
    symmetric_cbar='auto',   # bool | 'auto'
    title=None,              # str
    output_file=None,        # str — saves and closes
    axes=None,               # 3D matplotlib axes (projection='3d')
    figure=None,             # matplotlib Figure
    **kwargs                 # passed to plot_surf
)
# Returns: matplotlib Figure or PlotlySurfaceFigure
```

The `axes` parameter requires `projection='3d'` — this is the key to building multi-panel layouts in matplotlib. Nilearn The `engine='plotly'` option produces interactive HTML views Nilearn but cannot accept `axes`/`figure`. github Nilearn

plot_surf_roi() for parcellations

```python
nilearn.plotting.plot_surf_roi(
    surf_mesh=None, roi_map=None, bg_map=None,
    hemi='left', view=None, engine='matplotlib',
    cmap='gist_ncar',        # also accepts pandas DataFrame as BIDS LUT
    avg_method=None,         # defaults to 'median' for sharp parcel boundar
    threshold=None, alpha=None, bg_on_data=False,
    colorbar=True, axes=None, figure=None, output_file=None, **kwargs
)
```

The `cmap` parameter now accepts a pandas DataFrame or TSV path as a BIDS-compliant look-up table for custom parcellation coloring — a useful new feature. Nilearn

SurfaceImage: the new first-class data object

```python
from nilearn.surface import SurfaceImage
from nilearn.datasets import load_fsaverage, load_fsaverage_data

fsaverage_meshes = load_fsaverage("fsaverage5")
# Returns Bunch: 'pial'→PolyMesh, 'inflated'→PolyMesh, 'white_matter', 'sphe

# Project a volume to surface
stat_img = datasets.load_sample_motor_activation_image()
surface_image = SurfaceImage.from_volume(
    mesh=fsaverage_meshes["pial"], volume_img=stat_img
)
# Access parts: surface_image.data.parts["left"] → ndarray
```

fetch_surf_fsaverage() mesh keys (legacy, still functional)

The function returns a Bunch with string file paths: `pial_left`, `pial_right`, `infl_left`, `infl_right`, `white_left`, `white_right`, `sphere_left`, `sphere_right`, `flat_left`, `flat_right`, `sulc_left`, `sulc_right`, `curv_left`, `curv_right`. Nilearn
Mesh options: `'fsaverage3'` (642 nodes), `'fsaverage4'` (2562), `'fsaverage5'` (10242), `'fsaverage6'` (40962), `'fsaverage'` (163842). Nilearn

4-view panel — two approaches

Quick (volume input) using `plot_img_on_surf`:

```python
from nilearn.plotting import plot_img_on_surf
fig, axes = plot_img_on_surf(
    stat_map=stat_img, surf_mesh='fsaverage5',
    views=['lateral', 'medial'], hemispheres=['left', 'right'],
    threshold=1.0, cmap='cold_hot', inflate=True,
    colorbar=True, bg_on_data=True
)
fig.savefig('4view.png', dpi=300, bbox_inches='tight')
```

Manual subplot control for full customization:

```python
import matplotlib.pyplot as plt
fig, axes = plt.subplots(2, 2, subplot_kw={"projection": "3d"}, figsize=(12,

for ax, (view, hemi) in zip(axes.ravel(),
    [("lateral","left"),("medial","left"),("lateral","right"),("medial","rig
    plot_surf_stat_map(
        surf_mesh=fsaverage_meshes["inflated"], stat_map=surface_image,
        hemi=hemi, view=view, bg_map=sulcal, bg_on_data=True,
        threshold=1.0, cmap="cold_hot", colorbar=False,
        axes=ax, figure=fig, engine="matplotlib"
    )
fig.savefig('4view_manual.png', dpi=300, bbox_inches='tight')
```

---

3. nilearn connectome and glass brain functions share a display-object architecture

All nilearn volumetric plotting functions return **display objects** (`OrthoProjector`, `XZProjector`, etc.) with methods like Nilearn `add_graph()`, `add_markers()`, `add_overlay()`, and `add_contours()`. Nilearn This is the mechanism for compositing layers.

plot_connectome() — exact signature

```python
nilearn.plotting.plot_connectome(
    adjacency_matrix,          # ndarray (n, n) — symmetric for undirected
    node_coords,               # ndarray (n, 3) — MNI coordinates
    node_color='auto',         # color | sequence | 'auto'
    node_size=50,              # scalar | array-like (points²)
    edge_cmap='RdBu_r',       # colormap
    edge_vmin=None, edge_vmax=None,  # float
    edge_threshold=None,       # None | float | str (e.g. "90%")
    output_file=None,          # str
    display_mode='ortho',      # 'ortho','x','y','z','lzr','lyrz', etc.
    figure=None, axes=None,
    title=None, annotate=True, black_bg=False, alpha=0.7,
    edge_kwargs=None,          # dict → matplotlib Line2D
    node_kwargs=None,          # dict → plt.scatter
    colorbar=True, radiological=False
)
```

`edge_threshold` **behavior is counterintuitive:** `"90%"` shows only edges above the **90th percentile** (the top 10% strongest connections), not the top 90%. (github) A float value shows edges with `value > threshold`. The `display_mode` options include `'ortho'`, `'x'`, `'y'`, `'z'`, `'l'`, `'r'`, `'lr'`, `'lzr'`, `'lyr'`, `'lzry'`, `'lyrz'`. (Nilearn)

plot_markers() for node-only displays

```python
nilearn.plotting.plot_markers(
    node_values,               # array (n,) — values for color coding
    node_coords,               # ndarray (n, 3)
    node_size='auto',          # scalar | array | 'auto'
    node_cmap='gray',          # colormap
    node_vmin=None, node_vmax=None,
    node_threshold=None,
    alpha=0.7, display_mode='ortho',
    colorbar=True, node_kwargs=None, **kwargs
)
```

plot_glass_brain() with stat maps

```python
nilearn.plotting.plot_glass_brain(
    stat_map_img,              # Niimg-like | None
    threshold='auto',          # float | 'auto' (80th percentile) | None
    display_mode='ortho', colorbar=True,
    cmap=None, alpha=0.7,
    plot_abs=True,             # True=max intensity of |val|; False=show sig
    vmin=None, vmax=None,
    symmetric_cbar='auto',
    radiological=False,
    transparency=None,         # Niimg-like (added v0.12.0)
    output_file=None, **kwargs
)
```

Compositing glass brain + connectome via display objects

```python
from nilearn import plotting, datasets

stat_img = datasets.load_sample_motor_activation_image()
display = plotting.plot_glass_brain(
    stat_img, threshold=3, display_mode='z', colorbar=False, alpha=0.3
)
# Overlay connectome on the SAME display:
display.add_graph(
    adjacency_matrix, node_coords,
    node_color='black', node_size=15,
    edge_threshold="95%", edge_cmap='bwr',
    edge_kwargs={'linewidth': 1.5}
)
plotting.show()
```

`plot_connectome_strength()` **no longer exists** in the public API. GitHub Replace with `plot_markers(np.sum(np.abs(adj), axis=0), coords, node_cmap='YlOrRd')`.

---

4. Network and graph visualization spans four complementary tools

netgraph (v4.13.2) for publication-quality static graphs

netgraph's `Graph()` constructor accepts edges as a list, adjacency matrix, or networkx/igraph graph. GitHub The key parameter for brain visualization is `node_layout` — when passed a **dict of** `{node: (x, y)}`, it uses those exact positions:

```python
from netgraph import Graph

g = Graph(
    edges,                              # list of (u, v, weight) tuples
    node_layout=node_positions,         # dict {node_id: (x, y)}
    node_color=network_colors,          # dict {node_id: color}
    node_size=8,                        # float or dict
    node_labels=True,
    edge_layout='curved',              # 'straight' | 'curved' | 'arc' | 'bu
    edge_layout_kwargs=dict(k=0.02),
    edge_width=2.0,                    # float or dict
    edge_color='#555555',
    edge_cmap='RdGy',                  # auto-maps weights when weighted
    ax=ax                              # matplotlib axes
)
```

mne-connectivity (v0.7.0) for chord diagrams

The canonical function is `mne_connectivity.viz.plot_connectivity_circle()`. MNE Use `mne.viz.circular_layout()` to compute node angles with network group boundaries:

```python
from mne.viz import circular_layout
from mne_connectivity.viz import plot_connectivity_circle

node_angles = circular_layout(
    node_names, node_order, start_pos=90, group_boundaries=[0, 4, 8]
)
fig, ax = plot_connectivity_circle(
    con,                    # (n, n) matrix or 1D array
    node_names,             # list of str
    n_lines=20,             # show only N strongest
    node_angles=node_angles,
    node_colors=colors,     # list of colors per node
    colormap='RdYlBu_r',
    linewidth=2,
    facecolor='black', textcolor='white',
    fontsize_names=10, fontsize_title=14,
    ax=ax, show=False
)
```

networkx (v3.6.1) with MNI-projected positions

Project 3D MNI coordinates to 2D by selecting axis pairs: **axial = (x, y)**, **sagittal = (y, z)**, **coronal = (x, z)**:

```python
import networkx as nx

pos = {roi: (coord[0], coord[1]) for roi, coord in mni_coords.items()}  # ax
nx.draw_networkx(
    G, pos=pos,
    node_color=[net_cmap[G.nodes[n]['network']] for n in G.nodes()],
    node_size=600,
    edge_color='gray',
    width=[G[u][v]['weight'] * 3 for u, v in G.edges()],
    font_size=7, font_weight='bold', ax=ax
)
```

Connectivity matrix heatmaps with network ordering

```python
import seaborn as sns
import matplotlib.patches as patches

fig, ax = plt.subplots(figsize=(10, 9))
sns.heatmap(con_matrix, xticklabels=roi_names, yticklabels=roi_names,
            cmap='RdBu_r', center=0, square=True, ax=ax,
            cbar_kws={'label': 'Connectivity (r)', 'shrink': 0.8})

# Add network boundary lines
boundary = 0
for net_name, net_size in network_sizes.items():
    ax.axhline(y=boundary, color='white', linewidth=2)
    ax.axvline(x=boundary, color='white', linewidth=2)
    boundary += net_size
```

Alternatively, `sns.clustermap(df, row_colors=color_series, col_colors=color_series, row_cluster=False, col_cluster=False)` provides automatic network color sidebars without manual patching.

---

5. PyVista v0.47.1 enables full 3D control with off-screen rendering

Loading FreeSurfer surfaces — use make_tri_mesh

```python
import nibabel as nib
import pyvista as pv

vertices, faces = nib.freesurfer.read_geometry('/path/to/lh.pial')
mesh = pv.make_tri_mesh(vertices, faces)  # NO face padding needed
```

Critical gotcha: passing an (M, 3) face array directly to `pv.PolyData(verts, faces)` without VTK padding (prepending 3 to each row) will produce garbage or segfault. Always use `pv.make_tri_mesh()` PyVista (available since ~v0.43). If you must use `PolyData` directly: `vtk_faces = np.column_stack([np.full(n, 3), faces]).ravel().astype(np.int64)`.

Standard camera positions for neuroimaging

```python
CAMERA_POSITIONS = {
    'lateral_left':  [(-300, 0, 0), (0, 0, 0), (0, 0, 1)],
    'lateral_right': [( 300, 0, 0), (0, 0, 0), (0, 0, 1)],
    'medial_left':   [( 300, 0, 0), (0, 0, 0), (0, 0, 1)],  # LH viewed from
    'medial_right':  [(-300, 0, 0), (0, 0, 0), (0, 0, 1)],  # RH viewed from
    'dorsal':        [(0, 0,  300), (0, 0, 0), (0, 1, 0)],
    'ventral':       [(0, 0, -300), (0, 0, 0), (0, 1, 0)],
    'anterior':      [(0,  300, 0), (0, 0, 0), (0, 0, 1)],
    'posterior':     [(0, -300, 0), (0, 0, 0), (0, 0, 1)],
}
```

Each tuple is `(camera_location, focal_point, view_up)` in RAS coordinates.

Multi-subplot and off-screen rendering

```python
pv.start_xvfb()  # Linux headless only; macOS use PYVISTA_OFF_SCREEN=true

pl = pv.Plotter(
    shape=(2, 2), off_screen=True,
    window_size=[2400, 1800],
    image_scale=2              # → effective 4800×3600
)
pl.enable_anti_aliasing('ssaa')
pl.set_background('white')

pl.subplot(0, 0)
pl.add_mesh(mesh_l, scalars='stat_values', cmap='hot', clim=[0, 5],
            smooth_shading=True, show_scalar_bar=True,
            scalar_bar_args={'title': 'T-stat', 'vertical': True})
pl.camera_position = CAMERA_POSITIONS['lateral_left']

# ... repeat for (0,1), (1,0), (1,1) ...

pl.show(screenshot='brain_4view_3d.png')
pl.close()  # free GPU resources
```

Connectome spheres and tubes in 3D

```python
# ROI spheres
for x, y, z in roi_coords:
    sphere = pv.Sphere(radius=5, center=(x, y, z))
    pl.add_mesh(sphere, color='red', opacity=0.8)

# Edge tubes (radius scaled by weight)
for (p1, p2, weight) in connections:
    tube = pv.Tube(pointa=p1, pointb=p2, radius=weight * 2,
                   n_sides=20, capping=True)
    pl.add_mesh(tube, color='steelblue', opacity=0.7)
```

Subcortical structures from FreeSurfer aseg

```python
from skimage import measure
from scipy.ndimage import gaussian_filter

aseg_img = nib.load('mri/aseg.mgz')
aseg_data = aseg_img.get_fdata()
affine = aseg_img.affine

LABELS = {'Left-Hippocampus': 17, 'Right-Hippocampus': 53,
          'Left-Thalamus': 10, 'Right-Thalamus': 49,
          'Left-Caudate': 11, 'Left-Amygdala': 18}

def extract_structure(aseg_data, label_id, affine):
    binary = gaussian_filter((aseg_data == label_id).astype(float), sigma=0.
    verts, faces, normals, _ = measure.marching_cubes(binary, level=0.5)
    verts_world = (affine @ np.column_stack([verts, np.ones(len(verts))]).T)
    return pv.make_tri_mesh(verts_world, faces)

pl.add_mesh(cortex_mesh, color='white', opacity=0.15, smooth_shading=True)
for name, label_id in LABELS.items():
    struct = extract_structure(aseg_data, label_id, affine).smooth(n_iter=30
    pl.add_mesh(struct, color=STRUCTURE_COLORS[name], smooth_shading=True)
```

---

6. Publication standards converge on 300 DPI, Arial 7pt, and RGB output

Journal figure specifications (2025–2026)

Specification	Nature Neuroscience	NeuroImage	Brain (OUP)	PNAS
Single column	89 mm (3.50 in)	~90 mm	90 mm	87 mm (3.43 in)
Double column	183 mm (7.20 in)	~190 mm	185 mm	178 mm (7.0 in)
Photo DPI	300	300	300	300
Line art DPI	300 (vector preferred)	1000	600–1200	1000–1200
Format	TIFF/EPS/PDF	TIFF/EPS/PDF	TIFF preferred	TIFF/EPS/PDF
Color mode	RGB	RGB or CMYK	RGB	RGB required
Font	Arial/Helvetica 5–7pt	Arial 7pt min	Arial/Helvetica	Arial 6–8pt
Panel labels	**lowercase bold (a, b, c)**	a/A flexible	Uppercase (A, B)	**UPPERCASE bold (A, B, C)**

Practical export template for your module:

```python
EXPORT_SETTINGS = {
    'review':      {'dpi': 300, 'format': 'png'},
    'final':       {'dpi': 600, 'format': 'tiff'},
    'vector':      {'dpi': 300, 'format': 'pdf'},
    'single_col':  {'width_in': 3.46},   # ~88mm
    'double_col':  {'width_in': 7.09},   # ~180mm
    'font':        'Arial',
    'fontsize':    {'label': 8, 'title': 10, 'tick': 7, 'panel': 12},
}
```

Colormap selection rules

Sequential data (activation magnitude, cortical thickness, p-values): use **viridis** (default gold standard, fully colorblind-safe), **cividis** (maximum CVD safety), **inferno** or **plasma** for high contrast on dark backgrounds.

Diverging data (t-statistics, z-scores, correlation, left–right asymmetry): use **RdBu_r** (most common in neuroimaging), **PuOr** (better colorblind alternative), **coolwarm** (less saturated), or Crameri's **vik** / **berlin** (perceptually uniform, CVD-optimized).

Never use jet/rainbow — it creates false perceptual artifacts and fails ~8% of male viewers with deuteranopia. The key references are Crameri et al. (2020) *Nature Communications* 11:5444 and Pernet et al. (2021) *NeuroImage* 245:118628.

Yeo-7 canonical network colors (exact hex codes)

#	Network	RGB	Hex
1	Visual	120, 18, 134	**#781286**
2	Somatomotor	70, 130, 180	**#4682B4**
3	Dorsal Attention	0, 118, 14	**#00760E**
4	Ventral Attention	196, 58, 250	**#C43AFA**
5	Limbic	220, 248, 164	**#DCF8A4**
6	Frontoparietal	230, 148, 34	**#E69422**
7	Default Mode	205, 62, 78	**#CD3E4E**

Schaefer parcellations inherit these same network colors. The lookup tables are available at `github.com/ThomasYeoLab/CBIG/stable_projects/brain_parcellation/Schaefer2018_LocalGlobal/`.

---

7. Recipes for every figure type in brainplots.py

These recipes combine the APIs above into the specific figure types your module needs.

Vertex-wise statistical maps on inflated surfaces (4-view)

Use **surfplot** as the primary engine — it produces the cleanest 4-view panels with minimal code:

```python
def plot_stat_surface(lh_data, rh_data, cmap='YlOrRd_r', color_range=None,
                      cbar_label='', outline_parcellation=True, dpi=400):
    surfaces = fetch_fslr()
    lh, rh = surfaces['inflated']
    p = Plot(lh, rh, layout='grid', views=['lateral', 'medial'], zoom=1.5)
    p.add_layer({'left': surfaces['sulc'][0], 'right': surfaces['sulc'][1]},
                cmap='binary_r', cbar=False)
    p.add_layer({'left': lh_data, 'right': rh_data}, cmap=cmap,
                color_range=color_range, cbar_label=cbar_label)
    if outline_parcellation:
        lh_parc, rh_parc = load_parcellation('schaefer')
        p.add_layer({'left': lh_parc, 'right': rh_parc},
                    cmap='gray', as_outline=True, cbar=False)
    return p.build(figsize=(8, 6), cbar_kws={'fontsize': 8, 'shrink': 0.2})
```

ROI-based heatmaps (Schaefer/Yeo parcels colored by value)

Map scalar values to each parcel, then use surfplot with `zero_transparent=True`:

```python
def plot_parcel_values(parcel_values, parcellation_array, cmap='RdBu_r'):
    """parcel_values: dict {parcel_id: float} or array indexed by parcel lab
    vertex_data = np.zeros(len(parcellation_array))
    for parcel_id, value in parcel_values.items():
        vertex_data[parcellation_array == parcel_id] = value
    # Then use surfplot add_layer with this vertex_data
```

Asymmetry maps with diverging colormaps

```python
def plot_asymmetry(lh_data, rh_data, cmap='PuOr', vlim=None):
    """Compute and display L-R asymmetry on left hemisphere surface."""
    asymmetry = lh_data - rh_data
    if vlim is None:
        vlim = np.max(np.abs(asymmetry))
    p = Plot(lh, layout='row', views=['lateral', 'medial'])
    p.add_layer(asymmetry, cmap=cmap, color_range=(-vlim, vlim),
                cbar_label='L − R asymmetry')
    return p.build()
```

Connectome on glass brain (nodes + edges, thresholded)

```python
def plot_connectome_glass(adjacency_matrix, coords, threshold_pct="90%",
                          stat_map=None, display_mode='lzr'):
    if stat_map is not None:
        display = plotting.plot_glass_brain(
            stat_map, threshold=3, display_mode=display_mode,
            colorbar=False, alpha=0.3
        )
        display.add_graph(adjacency_matrix, coords,
                          edge_threshold=threshold_pct, edge_cmap='bwr',
                          node_size=20, node_color='black')
    else:
        display = plotting.plot_connectome(
            adjacency_matrix, coords,
            edge_threshold=threshold_pct, edge_cmap='RdBu_r',
            node_size=40, display_mode=display_mode, colorbar=True
        )
    return display
```

Network-level radar plots with Yeo-7 colors

```python
def plot_network_radar(network_values, network_names=None):
    """Radar/spider plot with one spoke per Yeo network."""
    YEO_COLORS = ['#781286','#4682B4','#00760E','#C43AFA',
                  '#DCF8A4','#E69422','#CD3E4E']
    if network_names is None:
        network_names = ['Vis','SM','DA','VA','Lim','FP','DMN']
    angles = np.linspace(0, 2*np.pi, len(network_names), endpoint=False).tol
    angles += angles[:1]
    values = list(network_values) + [network_values[0]]

    fig, ax = plt.subplots(figsize=(6, 6), subplot_kw=dict(polar=True))
    ax.plot(angles, values, 'o-', linewidth=2, color='#333')
    ax.fill(angles, values, alpha=0.15, color='#333')
    for i, (angle, val) in enumerate(zip(angles[:-1], values[:-1])):
        ax.plot(angle, val, 'o', color=YEO_COLORS[i], markersize=12, zorder=
    ax.set_xticks(angles[:-1])
    ax.set_xticklabels(network_names, fontsize=9, fontweight='bold')
    return fig
```

Graph metrics on brain (node size = centrality, edge color = weight)

```python
def plot_graph_metrics(G, coords, centrality_values, display_mode='lzr'):
    sizes = (centrality_values / centrality_values.max()) * 200 + 20
    display = plotting.plot_connectome(
        nx.to_numpy_array(G), coords,
        node_size=sizes,
        node_color=centrality_values,
        edge_threshold="85%",
        edge_cmap='coolwarm',
        display_mode=display_mode, colorbar=True,
        node_kwargs={'cmap': 'YlOrRd', 'edgecolors': 'black', 'linewidths':
    )
    return display
```

Multi-panel composite figures

```python
def composite_figure(brain_fig, stat_axes_func, n_brain_panels=1):
    """Combine brain views with statistical plots using gridspec."""
    from matplotlib.gridspec import GridSpec
    fig = plt.figure(figsize=(14, 8))
    gs = GridSpec(2, 3, figure=fig, width_ratios=[2, 1, 1], hspace=0.3, wspa

    # Brain panel (top-left, spans 2 rows)
    ax_brain = fig.add_subplot(gs[:, 0])
    # ... embed brain image via ax_brain.imshow(brain_screenshot) ...

    # Statistical panels
    ax_bar = fig.add_subplot(gs[0, 1])
    ax_scatter = fig.add_subplot(gs[0, 2])
    ax_matrix = fig.add_subplot(gs[1, 1:])
    return fig, {'brain': ax_brain, 'bar': ax_bar,
                 'scatter': ax_scatter, 'matrix': ax_matrix}
```

Key deprecations and gotchas to encode in brainplots.py

- **nilearn** `experimental.surface` moved to `nilearn.surface` in v0.11.0 — import from the new location
- **nilearn** `fetch_surf_fsaverage()` returns file paths; `load_fsaverage()` returns PolyMesh objects — use the latter for new code
- **surfplot** `add_fslr_medial_wall()` is required when data has 59,412 vertices (Schaefer parcellations without medial wall) — pad to 64,984
- **PyVista** `pv.PolyData(verts, faces)` without VTK padding produces segfaults — always use `pv.make_tri_mesh()`
- **nilearn** `plot_connectome_strength()` no longer exists — compute node strength manually and use `plot_markers()`
- **matplotlib 3.6+** broke surfplot colorbars — fixed in surfplot v0.2.0; pin surfplot ⩾ 0.2.0
- **PyVista off-screen on macOS** requires `PYVISTA_OFF_SCREEN=true` environment variable, not `start_xvfb()`
- **nibabel** `read_geometry()` returns FreeSurfer surface RAS coordinates, not scanner RAS — additional transforms may be needed for MNI overlay alignment

Conclusion

The Python neuroimaging visualization ecosystem in 2025–2026 is mature but fragmented across **three tiers of complexity**: surfplot for rapid 4-view publication panels (~5 lines of code), nilearn for glass brain/connectome compositing with its display-object architecture, and PyVista for full 3D camera control and subcortical rendering. A well-designed `brainplots.py` module should wrap all three behind a unified interface that enforces **300+ DPI, Arial 7–8pt fonts, and colorblind-safe defaults** (viridis for sequential, RdBu_r or PuOr for diverging). The biggest practical risk is vertex count mismatches between parcellations and surfaces — defensive checks against the known counts (64,984 for fsLR 32k, 10,242 for fsaverage5) will prevent the most common silent failures.